CSA0619-Design and Analysis of Algorithm

CO4_AT2_Industry-based Problem

Name: Preethika A
Reg No: 192511040

Q28. Online Advertising Budget Allocation:

CODE:

```
main.py
def hybrid_knapsack(campaigns, budget):
    campaigns.sort(
            key=lambda x: x[2] / x[1],
            reverse=True
    )
    selected = []
    remaining_budget = budget
    for campaign in campaigns:
        name, cost, revenue = campaign
        if cost <= remaining_budget:
            selected.append(campaign)
            remaining_budget -= cost
    n = len(campaigns)
    dp = [[0] * (budget + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        name, cost, revenue = campaigns[i - 1]
        for b in range(budget + 1):
            if cost <= b:
                dp[i][b] = max(
                    dp[i - 1][b],
                    revenue + dp[i - 1][b - cost]
                )
            else:
                dp[i][b] = dp[i - 1][b]

    max_revenue = dp[n][budget]

    return max_revenue
n = int(input("Enter number of campaigns: "))
budget = int(input("Enter advertising budget: "))
campaigns = []
for i in range(n):
    name = input(f"Enter campaign name {i + 1}: ")
    cost = int(input("Enter cost: "))
    revenue = int(input("Enter expected revenue: "))
    campaigns.append((name, cost, revenue))
```

OUTPUT:

```
27        for i in range(1, n + 1):
Enter number of campaigns: 4
Enter advertising budget: 10000
Enter campaign name 1: A
Enter cost: 2000
Enter expected revenue: 3000
Enter campaign name 2: B
Enter cost: 3000
Enter expected revenue: 5000
Enter campaign name 3: C
Enter cost: 4000
Enter expected revenue: 6000
Enter campaign name 4: D
Enter cost: 5000
Enter expected revenue: 7000
Maximum Revenue = 15000


...Program finished with exit code 0
Press ENTER to exit console.
```

Dynamic Programming :

```
dp[i][b] = max(
    dp[i - 1][b],
    revenue + dp[i - 1][b - cost]
)
```

Time Complexity:

O(nB)

Space Complexity:

O(nB)

(where:
n = number of campaigns
B = advertising budget)